

Thermodynamic EoS — Distillation Simulation (MATLAB)

main_dist.m

% This is the numerical solution of the system depicted in Fig. 15.6 (pp. 568) of
% Henley, E. J. and Seader, J. D., "Equilibrium-Stage Separation Operations
% in Chemical Engineering", 1981, John Wiley & Sons.

```
clear all; clc; format compact
```

```
load x0 % x0 = [x y L V T]
```

```
parameters_dist
```

```
N = 16; C = 5;
```

```
% States
```

```
% x = 0.25*ones(N,C);
```

```
% y = 0.25*ones(N,C);
```

```
% L = 100*ones(N,1);
```

```
% V = 100*ones(N,1);
```

```
% T = 150*ones(N,1);
```

```
% x0 = [x y L V T];
```

```
% Manipulations
```

```
P(1) = 238; % psia
```

```
P(2) = P(1) + 2;
```

```
P(3:N-1) = P(2) + DP*(3-2:N-1-2);
```

```
P(N) = P(N-1) + 0.4;
```

```
P = P/14.5038; % bar
```

```
U(1:N) = zeros(N,1); % kmole/h (from Aspen Plus simulation)
```

```
U(1) = 2.268;
```

```
U(3) = 1.361;
```

```
W(1:N) = zeros(N,1); % kmole/h (from Aspen Plus simulation)
```

```
W(13) = 16.783;
```

```
Q(1:N) = zeros(N,1); % Btu/h
```

```
Q(1) = 900274.3;
```

```
Q(3) = 200000.0;
```

```
Q(N) = -1566538.18;
```

```
Q = Q/9.47831e-4/1e3; % kJ/h
```

```
u = [P U W Q]';
```

```
% Disturbances
```

```
z = zeros(N,C);
```

```
z(6,:) = [2.5 14.0 19.0 5.0 0.5]/sum([2.5 14.0 19.0 5.0 0.5]);
```

```
z(9,:) = [0.5 6.0 18.0 30.0 4.5]/sum([0.5 6.0 18.0 30.0 4.5]);
```

```
F = zeros(N,1); % kmole/h (from Aspen Plus simulation)
```

```
F(6) = 18.597;
```

```
F(9) = 26.762;
```

```
PF = zeros(N,1); % psia
```

```
PF(6) = 300;
```

```
PF(9) = 275;
```

```
PF = PF/14.5038; % bar
```

```
TF = zeros(N,1); % F
```

```
TF(6) = 170;
```

```
TF(9) = 230;
```

```
TF = 1/1.8*(TF - 32); % C
```

```

HF      = zeros(N,1);
[HF(6),~,~,~] = feed_dist(F(6),z(6,:),TF(6),PF(6));
[HF(9),~,~,~] = feed_dist(F(9),z(9,:),TF(9),PF(9));

d      = [z F HF];

options = optimset('Display','iter','Diagnostics','on','Algorithm','interior-point');
r = fmincon('1',x0,[],[],[],[],[],[],@nonlcon_dist,options,u,d,N,C);

```

feed_dist.m

```
function [HF,psi,x,y] = feed_dist(F,z,TF,PF)

parameters_dist
c = length(z); % c includes all species
options = optimset('Display','iter','Diagnostics','on','Algorithm','interior-point');

% Bubble point calculation
xB = z'; % Bubble point requirement (together with PF)
yB = xB; % Initial guess
TB = TF; % Initial guess
x0 = [yB; TB];
r = fmincon('1',x0,[],[],[],[],[],[],@gB,options,xB,PF,c);
yB = r(1:c);
TB = r(end);

% Dew point calculation
yD = z'; % Dew point requirement (together with PF)
xD = yD; % Initial guess
TD = TF; % Initial guess
x0 = [xD; TD];
r = fmincon('1',x0,[],[],[],[],[],[],@gD,options,yD,PF,c);
xD = r(1:c);
TD = r(end);

if TF <= TB
    psi = 0;
    x = xB;
    y = yB;
    [~,~,~,~,~,HF,~,~] = prsrk(x,PF*1e5,TF+273.15,pc,Tc,w,k,cpig,DHf,DGf,'L',Thermo);
elseif TF >= TD
    psi = 1;
    x = xD;
    y = yD;
    [~,~,~,~,~,HF,~,~] = prsrk(y,PF*1e5,TF+273.15,pc,Tc,w,k,cpig,DHf,DGf,'V',Thermo);
else
    % Two-phase calculation
    x = z'; % Initial guess
    y = x; % Initial guess
    V = 0.5*F; % Initial guess
    L = V; % Initial guess
    x0 = [x; y; V; L]; d = [F; TF; PF; z'];
    r = fmincon('1',x0,[],[],[],[],[],[],@gF,options,d,c);
    x = r(1:c); y = r(c+1:2*c);
    [~,~,~,~,~,HL,~,~] = prsrk(x,PF*1e5,TF+273.15,pc,Tc,w,k,cpig,DHf,DGf,'L',Thermo);
    [~,~,~,~,~,HV,~,~] = prsrk(y,PF*1e5,TF+273.15,pc,Tc,w,k,cpig,DHf,DGf,'V',Thermo);
    V = r(end-1); L = r(end);
    HF = 1/F*(HV*V + HL*L); psi = r(end-1)/F;
end

function [c,ceq] = gB(x0,x,PF,c)

parameters_dist
y = x0(1:c);
T = x0(end);

 [~,~,phiL,~,~,~,~] = prsrk(x,PF*1e5,T+273.15,pc,Tc,w,k,cpig,DHf,DGf,'L',Thermo);
 [~,~,phiV,~,~,~,~] = prsrk(y,PF*1e5,T+273.15,pc,Tc,w,k,cpig,DHf,DGf,'V',Thermo);
K = phiL./phiV;

ceq = [y - K.*x; sum(y) - 1];
c = [];

function [c,ceq] = gD(x0,y,PF,c)
```

```

parameters_dist
x = x0(1:c);
T = x0(end);

[~,~,phiL,~,~,~,~] = prsrk(x,PF*1e5,T+273.15,pc,Tc,w,k,cpig,DHf,DGf,'L',Thermo);
[~,~,phiV,~,~,~,~] = prsrk(y,PF*1e5,T+273.15,pc,Tc,w,k,cpig,DHf,DGf,'V',Thermo);
K = phiL./phiV;

ceq = [y - K.*x; sum(x) - 1];
c = [];

function [c,ceq] = gF(x0,d,c)

parameters_dist
x = x0(1:c);
y = x0(c+1:2*c);
V = x0(end-1);
L = x0(end);

F = d(1);
TF = d(2);
PF = d(3);
z = d(4:end);

[~,~,phiL,~,~,~,~] = prsrk(x,PF*1e5,TF+273.15,pc,Tc,w,k,cpig,DHf,DGf,'L',Thermo);
[~,~,phiV,~,~,~,~] = prsrk(y,PF*1e5,TF+273.15,pc,Tc,w,k,cpig,DHf,DGf,'V',Thermo);
K = phiL./phiV;

ceq = [y - K.*x; F*z - V*y - L*x; sum(x) - 1; sum(y) - 1];
c = [];

```

model_dist.m

% Steady-state rigorous distillation model

% Thermodynamic calculations

HL = zeros(N,1); HV = HL; K = zeros(N,C);

for j = 1:N

 [~,~,phiL,~,~,HL(j),~,~] = prsrk(x(j,:),P(j)*1e5,T(j)+273.15,pc,Tc,w,k,cpig,DHf,DGf,'L',Thermo);

 [~,~,phiV,~,~,HV(j),~,~] = prsrk(y(j,:),P(j)*1e5,T(j)+273.15,pc,Tc,w,k,cpig,DHf,DGf,'V',Thermo);

 K(j,:) = phiL./phiV;

end

% Material balances (N*C)

j = 1; M(j,:) = V(j+1)*y(j+1,:) + F(j)*z(j,:) - (L(j) + U(j))*x(j,:) - (V(j) + W(j))*y(j,:);

j = 2:N-1; M(j,:) = diag(L(j-1))*x(j-1,:) + diag(V(j+1))*y(j+1,:) + diag(F(j))*z(j,:) - diag((L(j) + U(j)))*x(j,

j = N; M(j,:) = L(j-1)*x(j-1,:) + F(j)*z(j,:) - (L(j) + U(j))*x(j,:) - (V(j) + W(j))*y(j,:);

% Equilibrium relations (N*C)

E = y - K.*x;

% Mole fraction summations (2*N)

Sx = sum(x,2) - 1;

Sy = sum(y,2) - 1;

% Energy balances (N)

j = 1; H(j) = V(j+1)*HV(j+1) + F(j)*HF(j) - (L(j) + U(j))*HL(j) - (V(j) + W(j))*HV(j) - Q(j);

j = 2:N-1; H(j) = L(j-1).*HL(j-1) + V(j+1).*HV(j+1) + F(j).*HF(j) - (L(j) + U(j)).*HL(j) - (V(j) + W(j)).*HV(j)

j = N; H(j) = L(j-1)*HL(j-1) + F(j)*HF(j) - (L(j) + U(j))*HL(j) - (V(j) + W(j))*HV(j) - Q(j);

nonlcon_dist.m

```
function [c,ceq] = nonlcon_dist(x0,u,d,N,C)
```

```
parameters_dist
```

```
% States
```

```
x = x0(:,0*C+1:1*C);
```

```
y = x0(:,1*C+1:2*C);
```

```
L = x0(:,2*C+1);
```

```
V = x0(:,2*C+2);
```

```
T = x0(:,2*C+3);
```

```
% Manipulations
```

```
P = u(0*N+1:1*N);
```

```
U = u(1*N+1:2*N);
```

```
W = u(2*N+1:3*N);
```

```
Q = u(3*N+1:4*N);
```

```
% Disturbances
```

```
z = d(:,0*C+1:1*C);
```

```
F = d(:,1*C+1);
```

```
HF = d(:,1*C+2);
```

```
model_dist
```

```
ceq = [M(:); E(:); Sx(:); Sy(:); H(:)];
```

```
c = [];
```

parameters_dist.m

% Parameters

```
Thermo = 'PR'; % Thermodynamic package = {'PR', 'SRK', 'RK'}
DP      = 0.2; % Pressure drop from stage 2 to N-1
% Physical parameters
pc      = [709.8 616.410385 550.70829 487.326798 436.56359]'; % psia
pc      = pc*1e5/14.5038; % Pa
Tc      = [89.906 206.006002 305.618002 385.412001 453.47]'; % F
Tc      = (Tc - 32)/1.8 + 273.15; % K
w       = [0.1064 0.152 0.201 0.254 0.301]';
k(1,1) = 0;
k(1,2) = 1.1e-3;
k(1,3) = 9.6e-3;
k(1,4) = 7.8e-3;
k(1,5) = -0.01;
k(2,2) = 0;
k(2,3) = 3.3e-3;
k(2,4) = 0.0267;
k(2,5) = 7.0e-4;
k(3,3) = 0;
k(3,4) = 0.0174;
k(3,5) = -5.6e-3;
k(4,4) = 0;
k(4,5) = 0;
k(5,5) = 0;
k       = k + k';
cpig    = [40.33 51.92 71.34 88.05 104.40; 134.22 192.45 243.00 301.10 352.30; 1655.50 1626.50 1630.00 1650.20 1
           73.22 116.80 150.33 189.20 236.90; 752.87 723.60 730.42 747.60 761.60]'; % J/mol-K
DHf     = [-8.3820e+4 -1.0468e+5 -1.2579e+5 -1.4676e+5 -1.6694e+5]'; % J/mol
DGF     = [-3.1920e+4 -2.4290e+4 -1.6570e+4 -0.8650e+4 +0.0150e+4]'; % J/mol
```

prsrk.m

% Thermodynamic property calculations based on EoS

% Inputs:

% z = number of moles of all species (n x 1 column vector) [mol]

% p = pressure [Pa]

% T = temperature [K]

% pc = critical pressure of all components (n x 1 column vector)[Pa]

% Tc = critical temperature of all components (n x 1 column vector)[K]

% w = acentric factor of all components (n x 1 column vector)

% k = binary parameters (n x n symmetric matrix)

% cpig = ideal gas heat capacity coefficients (n x m symmetric matrix) [J/mol-K]

% DHf = standard enthalpy of formation for ideal gas (298.15 K and 1 atm) (n x 1 column vector) [J/mol]

% DGf = standard Gibbs energy of formation for ideal gas (298.15 K and 1 atm) (n x 1 column vector) [J/mol]

% ph = phase ('L' or 'V')

% flag = chosen EoS ('PR', 'SRK', or 'RK')

% Outputs:

% V = molar volume [m3/mol]

% Z = compressibility factor

% phi = fugacity coefficient

% A = Helmholtz energy [J/mol]

% S = entropy [J/(mol K)]

% H = enthalpy [J/mol]

% U = internal eenergy [J/mol]

% G = Gibbs energy [J/mol]

function [rho,Z,phi,A,S,H,U,G] = prsrk(z,p,T,pc,Tc,w,k,cpig,DHf,DGf,ph,flag)

z = z(:); % z is always a column vector

R = 8.314; % m3 Pa/(mol K) = J/mol-K

Tref = 298.15; % K (reference temperature)

Pref = 1e5; % Pa (reference pressure)

DSf = 1/T*(DHf - DGf); % standard entropy of formation [J/(mol K)]. Calculated at the given temperature, not t

switch flag

case{'PR'}

epsi = 1 - sqrt(2);

sigm = 1 + sqrt(2);

omeg = 0.07780;

psi = 0.45724;

mi = [0.37464 1.54226 0.26992];

m = [ones(length(w),1) w -w.^2]*mi';

case{'SRK'}

epsi = 0;

sigm = 1;

omeg = 0.08664;

psi = 0.42748;

mi = [0.480 1.574 0.176];

m = [ones(length(w),1) w -w.^2]*mi';

case{'RK'}

epsi = 0;

sigm = 1;

omeg = 0.08664;

psi = 0.42748;

m = ((T./Tc).^(-1/4) - 1)./(1 - (T./Tc).^0.5);

end

alfa = (1 + m.*(1 - (T./Tc).^0.5)).^2;

ai = psi*(R^2)*(Tc.^2)./pc.*alfa;

bi = omeg*R*Tc./pc;

Q = ((ai*ai').^0.5).*(1 - k);

a = z'*Q*z;

b = z'*bi;

dQdT = psi*(R^2)*(k - 1).*(Tc*Tc')./((pc*pc').^0.5).*(1/(2*T^0.5)).*...
((m./(Tc.^0.5)).*(alfa.^0.5)' + (alfa.^0.5)*(m./(Tc.^0.5))');


```

dadT = z'*dQdT*z;

% Coefficients of the EoS model equation
c(1) = 1;
c(2) = (epsi + sigm - 1)*b - R*T/p;
c(3) = (epsi*sigm - (epsi + sigm))*b^2 - R*T*(epsi + sigm)/p*b + a/p;
c(4) = -epsi*sigm*b^3 - R*T*epsi*sigm/p*b^2 - a*b/p;

% Roots
r = roots(c);
ir = abs(imag(r)); r(and(ir>0,ir<=1e-8)) = real(r(and(ir>0,ir<=1e-8))); % In case Matlab assigns tiny values for
for i = 1:length(r), index(i) = isreal(r(i)); end
r = r(index);
if upper(ph) == 'L'
    V = min(r);
else
    V = max(r);
end
rho = 1/V;
Z = p*V/(R*T);

u = epsi + sigm;
v = epsi*sigm;
Bs = b*p/(R*T);
AR = a/(b*(sqrt(u^2 - 4*v)))*log((2*Z + Bs*(u - sqrt(u^2 - 4*v)))/(2*Z + Bs*(u + sqrt(u^2 - 4*v)))) - ...
    R*T*log((Z - Bs)/Z) - R*T*log(V/(R*T/p));
SR = R*log((Z - Bs)/Z) + R*log(V/(R*T/p)) - 1/(b*(sqrt(u^2 - 4*v)))*dadT*...
    log((2*Z + Bs*(u - sqrt(u^2 - 4*v)))/(2*Z + Bs*(u + sqrt(u^2 - 4*v))));
HR = AR + T*SR + R*T*(Z - 1);

% From Aly, F. A. and Lee, L. L., "Self-Consistent Equations for
% Calculating the Ideal Gas Heat Capacity, Enthalpy, and Entropy",
% Fluid Phase Equilibria, Vol. 6, 1981, p. 169, we have:
% Cp = C1 + C2*((C3/T)/sinh(C3/T))^2 + C4*((C5/T)/cosh(C5/T))^2.
% But:
% Hig = z'*(DHf + int(Cp,T,Tref,T)).
% Sig = z'*(DSf + int(Cp/T,T,Tref,T) - R*ln(P/Pref)) - R*z'*ln(z).
Hig = z'*(DHf + cpig(:,1)*(T - Tref) + 2*cpig(:,2).*cpig(:,3).*(1./(exp(2*cpig(:,3)/T) - 1) - 1./(exp(2*cpig(:,3)/Tref) - 1)) ...
    + 2*cpig(:,4).*cpig(:,5).*(1./(exp(2*cpig(:,5)/T) + 1) - 1./(exp(2*cpig(:,5)/Tref) + 1))));
Sig = z'*(DSf + cpig(:,1)*log(T/Tref) - cpig(:,2).*log((exp(2*cpig(:,3)/T) - 1)./(exp(2*cpig(:,3)/Tref) - 1)) ...
    + cpig(:,4).*log((exp(2*cpig(:,5)/T) + 1)./(exp(2*cpig(:,5)/Tref) + 1)) + 2*(cpig(:,2).*cpig(:,3) - cpig(:,2).*cpig(:,3).*(1./(T*(exp(2*cpig(:,3)/T) - 1)) - 1./(Tref*(exp(2*cpig(:,3)/Tref) - 1))) ...
    + 2*cpig(:,4).*cpig(:,5).*(1./(T*(exp(2*cpig(:,5)/T) - 1)) - 1./(Tref*(exp(2*cpig(:,5)/Tref) - 1))) ...
    - R*log(p/Pref)) - R*z'*log(z);
abar = (2*z'*Q - a*ones(1,length(z)))'; bbar = bi;
phi = exp((Z - 1)*bbar/b - log((V - b)*Z/V) + (a/(b*R*T))/(epsi - sigm)*log((V + sigm*b)/(V + epsi*b)))*...
    (1 + abar/a - bbar/b));

H = Hig + HR;
S = Sig + SR;
U = H - p*V;
A = U - T*S;
G = H - T*S;

```

license.txt

Copyright (c) 2013, Antonio
All rights reserved.

Redistribution and use in source and binary forms, with or without modification, are permitted provided that the following conditions are met:

- * Redistributions of source code must retain the above copyright notice, this list of conditions and the following disclaimer.
- * Redistributions in binary form must reproduce the above copyright notice, this list of conditions and the following disclaimer in the documentation and/or other materials provided with the distribution

THIS SOFTWARE IS PROVIDED BY THE COPYRIGHT HOLDERS AND CONTRIBUTORS "AS IS" AND ANY EXPRESS OR IMPLIED WARRANTIES, INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR PURPOSE ARE DISCLAIMED. IN NO EVENT SHALL THE COPYRIGHT OWNER OR CONTRIBUTORS BE LIABLE FOR ANY DIRECT, INDIRECT, INCIDENTAL, SPECIAL, EXEMPLARY, OR CONSEQUENTIAL DAMAGES (INCLUDING, BUT NOT LIMITED TO, PROCUREMENT OF SUBSTITUTE GOODS OR SERVICES; LOSS OF USE, DATA, OR PROFITS; OR BUSINESS INTERRUPTION) HOWEVER CAUSED AND ON ANY THEORY OF LIABILITY, WHETHER IN CONTRACT, STRICT LIABILITY, OR TORT (INCLUDING NEGLIGENCE OR OTHERWISE) ARISING IN ANY WAY OUT OF THE USE OF THIS SOFTWARE, EVEN IF ADVISED OF THE POSSIBILITY OF SUCH DAMAGE.